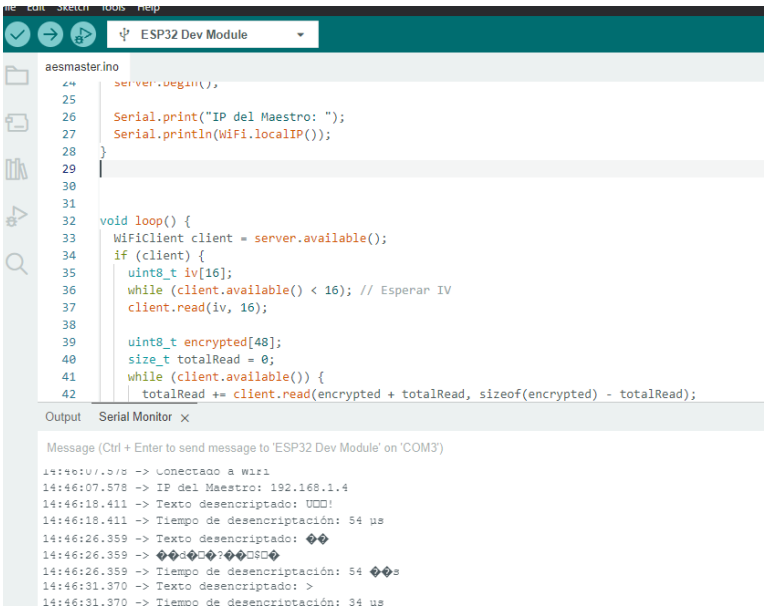
ANEXOS

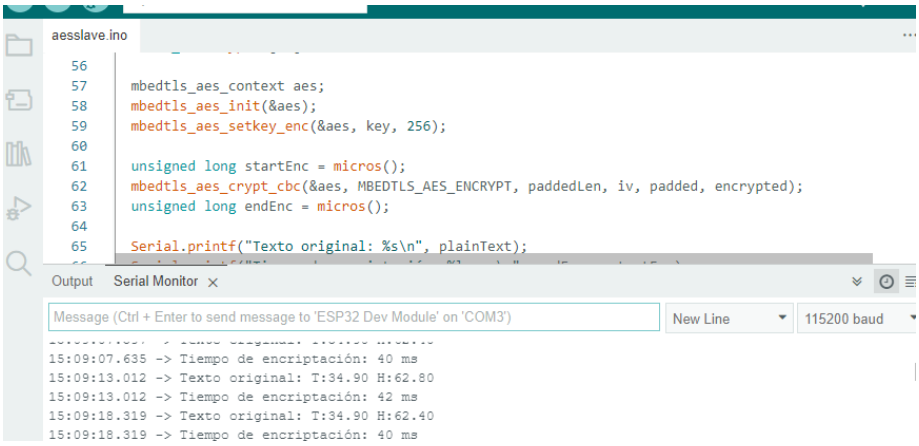
Recolección de métricas

Figura 1. Medición de tiempo de cifrado del algoritmo AES



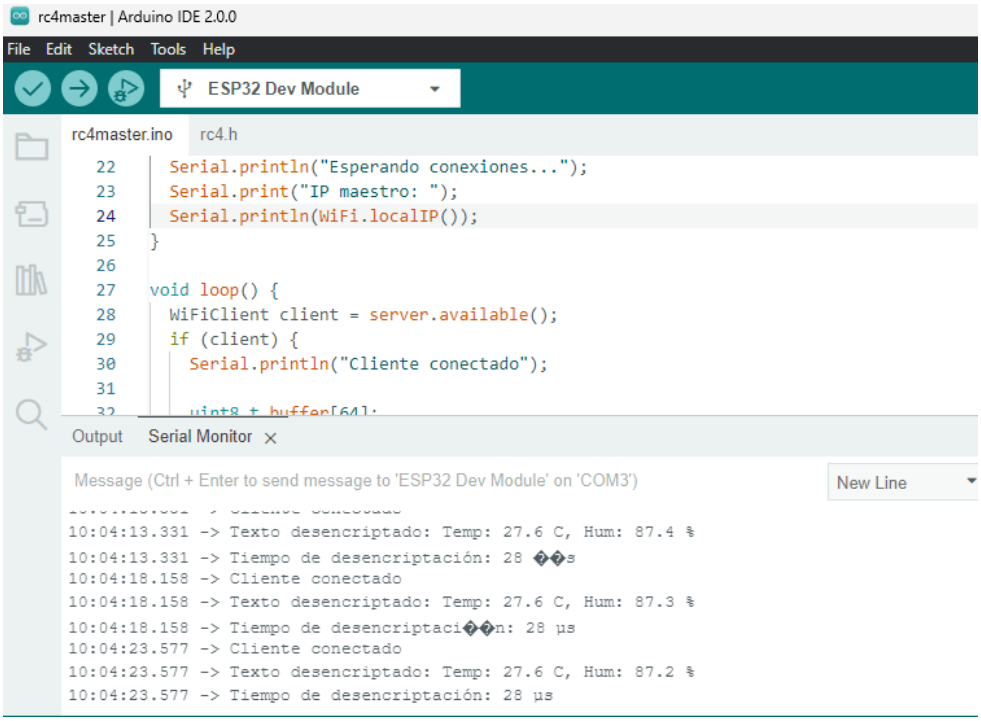
Elaborado por: Autor

Figura 2. Medición de tiempo de descifrado del algoritmo AES



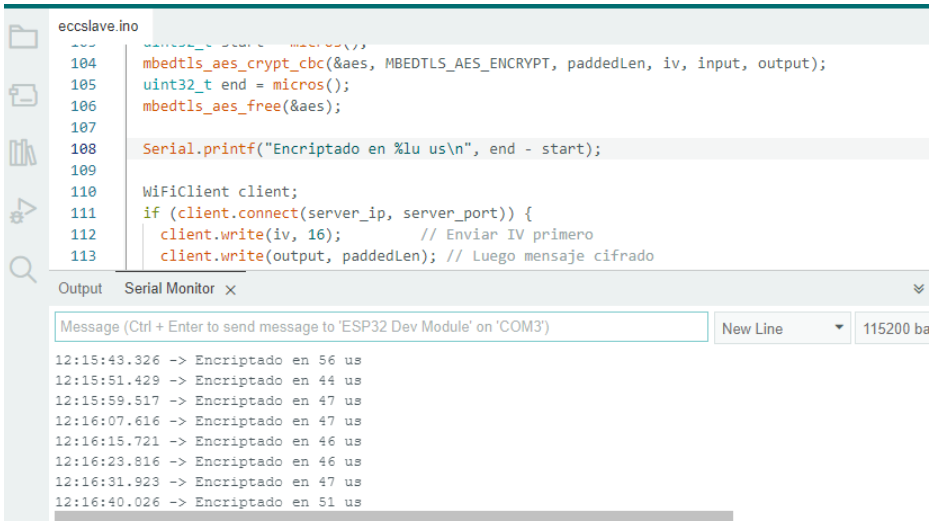
Elaborado por: Autor

Figura 3. Medición de tiempo de descifrado del algoritmo RC4



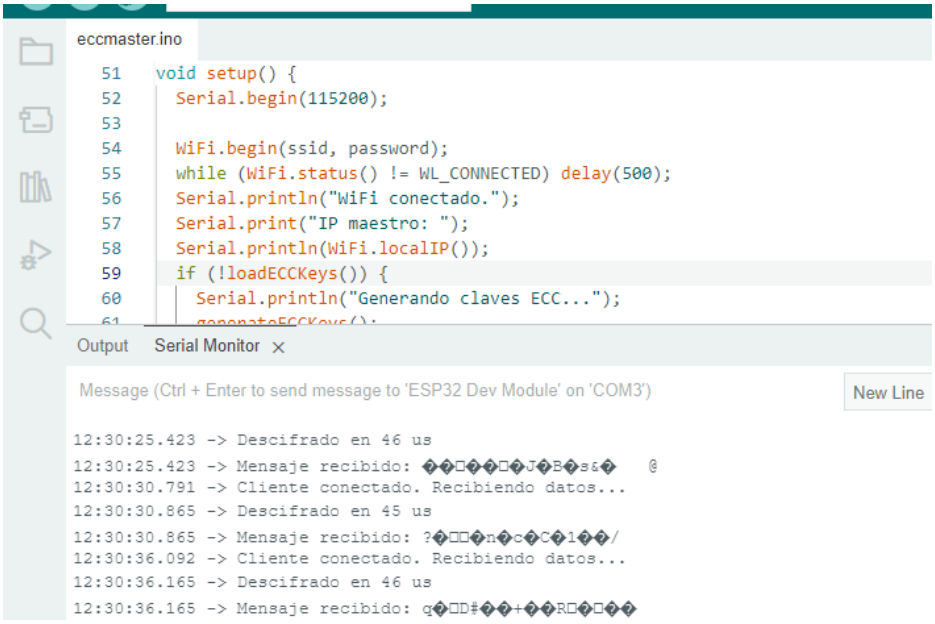
Elaborado por: Autor

Figura 4. Medición de tiempo de cifrado del algoritmo ECC



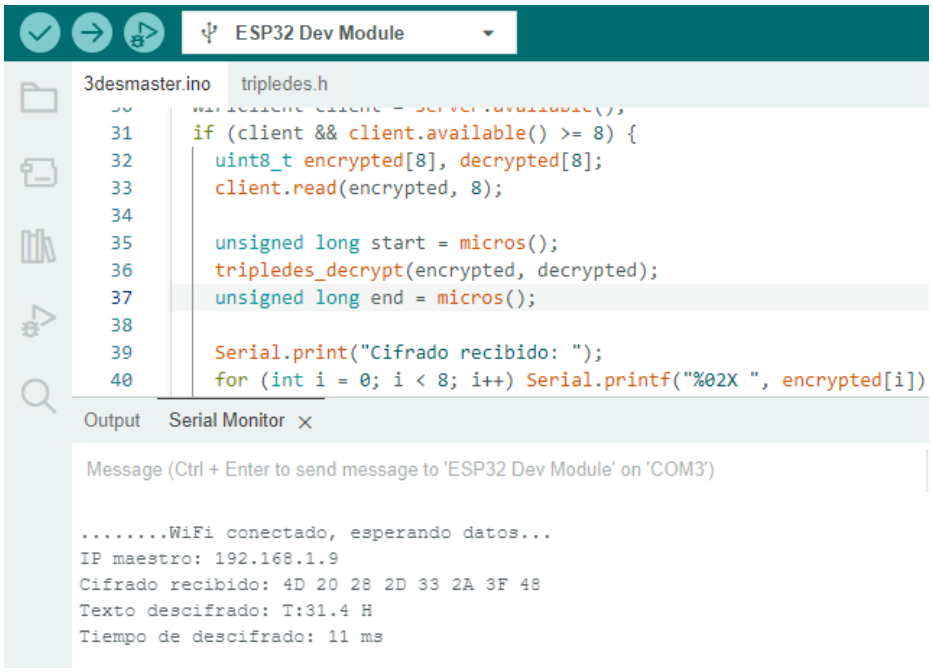
Elaborado por: Autor

Figura 5. Medición de tiempo de descifrado del algoritmo ECC



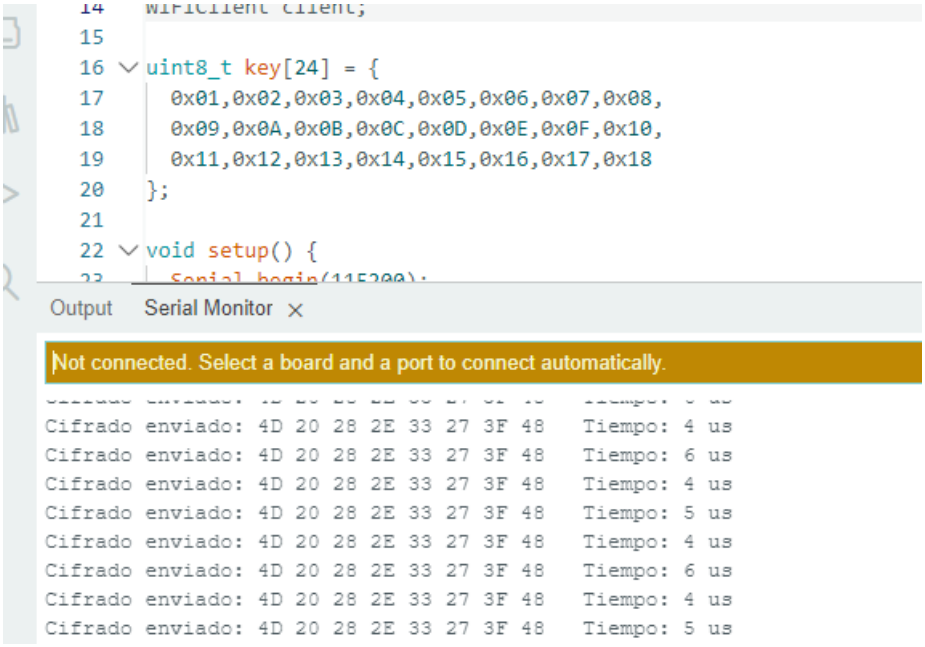
Elaborado por: Autor

Figura 6. Medición de tiempo de cifrado del algoritmo 3DES



Elaborado por: Autor

Figura 7. Medición de tiempo de descifrado del algoritmo 3DES



Elaborado por: Autor

Código fuente de los algoritmos de cifrado

Código del nodo sensor con el algoritmo AES

```
#include <WiFi.h>
#include <DHT.h>
#include "mbedtls/aes.h"
#include <string.h>

#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

const char* ssid = "";
const char* password = "";
const char* host = "192.168.1.6";  // IP del receptor
const int port = 1234;

uint8_t key[32] = {0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
  0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F,
  0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17,
  0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F};
uint8_t iv[16] = {1};  // IV estático para pruebas (puede ser aleatorio)
```

```
WiFiClient client;

void padPKCS7(uint8_t* input, size_t inputLen, uint8_t* output, size_t blockSize, size_t& outputLen) {
  uint8_t padLen = blockSize - (inputLen % blockSize);
  memcpy(output, input, inputLen);
  for (int i = 0; i < padLen; i++) output[inputLen + i] = padLen;
  outputLen = inputLen + padLen;
}

void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(1000);
  Serial.println("Conectado a WiFi");
}

void loop() {
  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  if (isnan(temp) || isnan(hum)) {
    Serial.println("Error al leer el sensor DHT22");
    delay(2000);
    return;
  }

  char plainText[32];
  snprintf(plainText, sizeof(plainText), "T:%.2f H:%.2f", temp, hum);

  uint8_t padded[48]; // Espacio suficiente para texto + padding
  size_t paddedLen;
  padPKCS7((uint8_t*)plainText, strlen(plainText), padded, 16, paddedLen);

  uint8_t encrypted[48];

  mbedtls_aes_context aes;
  mbedtls_aes_init(&aes);
  mbedtls_aes_setkey_enc(&aes, key, 256);

  unsigned long startEnc = micros();
```

```
  mbedtls_aes_crypt_cbc(&aes, MBEDTLS_AES_ENCRYPT, paddedLen, iv, padded, encrypted);
  unsigned long endEnc = micros();

  Serial.printf("Texto original: %s\n", plainText);
  Serial.printf("Tiempo de encriptación: %lu ms\n", endEnc - startEnc);
  if (client.connect(host, port)) {
    client.write(iv, 16);         // Enviar IV
    client.write(encrypted, paddedLen);  // Enviar datos cifrados
    client.stop();
  } else {
    Serial.println("Error al conectar con receptor");
  }
  delay(5000);
}
```

Código del nodo central con el algoritmo AES

```
#include <WiFi.h>
#include "mbedtls/aes.h"
#include <string.h>
const char* ssid = "";
const char* password = "";

WiFiServer server(1234);
uint8_t key[32] = {0};  // AES-256

void removePKCS7Padding(uint8_t* data, size_t& len) {
  uint8_t padLen = data[len - 1];
  if (padLen > 0 && padLen <= 16) {
    len -= padLen;
    data[len] = '\0';
  }
}
void setup() {
  Serial.begin(115200);

  WiFi.begin(ssid, password);
  Serial.print("Conectando a WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
```

```
  Serial.println("\nConectado a WiFi");
  server.begin();
  Serial.println("Servidor iniciado, esperando datos...");
}

void loop() {
  WiFiClient client = server.available();
  if (client) {
    uint8_t original_iv[16];
    while (client.available() < 16);
    client.read(original_iv, 16);
    uint8_t iv[16];
    memcpy(iv, original_iv, 16);

    uint8_t encrypted[64];
    size_t totalRead = 0;
    while (client.connected() && client.available()) {
      totalRead += client.read(encrypted + totalRead, sizeof(encrypted) - totalRead);
      if (totalRead >= sizeof(encrypted)) break;
    }

    if (totalRead % 16 != 0) {
      Serial.println("Error: longitud cifrada no es múltiplo de 16.");
      client.stop();
      return;
    }

    Serial.printf("Bytes cifrados recibidos: %d\n", totalRead);

    uint8_t decrypted[64];
    mbedtls_aes_context aes;
    mbedtls_aes_init(&aes);
    mbedtls_aes_setkey_dec(&aes, key, 256);

    unsigned long startDec = micros();
    mbedtls_aes_crypt_cbc(&aes, MBEDTLS_AES_DECRYPT, totalRead, iv, encrypted, decrypted);
    unsigned long endDec = micros();

    size_t len = totalRead;
    removePKCS7Padding(decrypted, len);

    Serial.printf("Texto desencriptado: %.*s\n", len, decrypted);
```

```
  Serial.printf("Tiempo de desencriptación: %lu µs (%.2f ms)\n", endDec - startDec, (endDec - startDec) / 1000.0);
 }
}
```

Código del nodo sensor con el algoritmo RC4

```
#include <WiFi.h>
#include <DHT.h>
#include "rc4.h"  //

const char* ssid = "";
const char* password = "";
const char* host = "192.168.1.6"; // IP del receptor
const int port = 1234;

#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

uint8_t key[24] = {0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,
 0x09,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F,0x10,
 0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18
};
const size_t keylen = strlen((char*)key);

void setup() {
 Serial.begin(115200);
 dht.begin();

 WiFi.begin(ssid, password);
 Serial.print("Conectando a WiFi");
 while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
 }
 Serial.println("\nConectado a WiFi");
}
void loop() {
 float temp = dht.readTemperature();
 float hum = dht.readHumidity();

 if (isnan(temp) || isnan(hum)) {
  Serial.println("Error al leer DHT22");
```

```
  delay(2000);
  return;
 }
 char payload[64];
 snprintf(payload, sizeof(payload), "Temp: %.1f C, Hum: %.1f %%", temp, hum);
 size_t len = strlen(payload);

 uint8_t encrypted[64];
 memcpy(encrypted, payload, len);

 unsigned long start = micros();
 RC4 rc4(key, keylen);
 rc4.crypt(encrypted, len);
 unsigned long end = micros();

 Serial.printf("Tiempo de encriptación: %lu µs\n", end - start);

 WiFiClient client;
 if (client.connect(host, port)) {
  client.write("RC4:", 4);  // Prefijo útil para Wireshark
  client.write(encrypted, len);
  client.stop();
  Serial.println("Datos enviados.");
 } else {
  Serial.println("Error al conectar.");
 }
 delay(5000);
}
```

Código del nodo central con el algoritmo RC4

```
#include <WiFi.h>
#include "rc4.h"

const char* ssid = "";
const char* password = "";

WiFiServer server(1234);

uint8_t key[] = "ClaveRC4Segura123";
const size_t keylen = strlen((char*)key);
void setup() {
 Serial.begin(115200);
```

```
  WiFi.begin(ssid, password);
  Serial.print("Conectando a WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nConectado a WiFi");
  server.begin();
  Serial.println("Esperando conexiones...");
  Serial.print("IP maestro: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  WiFiClient client = server.available();
  if (client) {
    Serial.println("Cliente conectado");

    uint8_t buffer[64];
    size_t totalRead = 0;

    while (client.connected() && client.available()) {
      totalRead += client.read(buffer + totalRead, sizeof(buffer) - totalRead);
      if (totalRead >= sizeof(buffer)) break;
    }
    if (totalRead < 5 || memcmp(buffer, "RC4:", 4) != 0) {
      Serial.println("Paquete inválido.");
      client.stop();
      return;
    }
    size_t cipherLen = totalRead - 4;
    uint8_t decrypted[64];
    memcpy(decrypted, buffer + 4, cipherLen);

    unsigned long start = micros();
    RC4 rc4(key, keylen);
    rc4.crypt(decrypted, cipherLen);
    unsigned long end = micros();

    decrypted[cipherLen] = '\0'; // Para imprimir como texto
    Serial.printf("Texto desencriptado: %s\n", decrypted);
    Serial.printf("Tiempo de desencriptación: %lu µs\n", end - start);
```

```
  client.stop();
 }
}
```

Código del nodo sensor con el algoritmo ECC

```
#include <WiFi.h>
#include <uECC.h>
#include <mbedtls/aes.h>
#include <DHT.h>
#include <Preferences.h>

#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

// WiFi
const char* ssid = "";
const char* password = "";
const char* server_ip = "192.168.1.6";
const uint16_t server_port = 1234;

// ECC
Preferences prefs;
uint8_t privateKey[32];
uint8_t publicKey[64];

// Clave pública del receptor (REEMPLAZA)
uint8_t peerPublicKey[64] = {
  0xA7, 0x63, 0xDA, 0x4E, 0x08, 0xB1, 0x19, 0xF3,
  0x2D, 0x7A, 0x6F, 0xED, 0x45, 0x0B, 0xC1, 0x35,
  0x74, 0xE5, 0x99, 0xAC, 0x8B, 0x9D, 0xF1, 0xC4,
  0x56, 0x3E, 0x11, 0x9C, 0x2F, 0xA0, 0xD8, 0x91,
  0xFE, 0xE3, 0x64, 0x12, 0x37, 0x58, 0xF6, 0x00,
  0x6C, 0x87, 0x92, 0x9B, 0xC5, 0xE2, 0x0F, 0x4A,
  0x31, 0xDE, 0xAB, 0x2C, 0x73, 0x08, 0x5D, 0x67,
  0x9E, 0x7F, 0x0D, 0xD9, 0xB0, 0xC2, 0x49, 0x55
};

void generateECCKeys() {
  const struct uECC_Curve_t* curve = uECC_secp256r1();
  uECC_make_key(publicKey, privateKey, curve);
```

```
  prefs.begin("ecc", false);
  prefs.putBytes("priv", privateKey, 32);
  prefs.putBytes("pub", publicKey, 64);
  prefs.end();
}

bool loadECCKeys() {
  prefs.begin("ecc", true);
  if (prefs.isKey("priv") && prefs.isKey("pub")) {
    prefs.getBytes("priv", privateKey, 32);
    prefs.getBytes("pub", publicKey, 64);
    prefs.end();
    return true;
  }
  prefs.end();
  return false;
}

void setup() {
  Serial.begin(115200);
  dht.begin();

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("WiFi conectado.");

  if (!loadECCKeys()) {
    Serial.println("Generando claves ECC...");
    generateECCKeys();
  } else {
    Serial.println("Claves ECC cargadas.");
  }

  Serial.println("Clave pública del emisor:");
  for (int i = 0; i < 64; i++) Serial.printf("%02X", publicKey[i]);
  Serial.println();
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
```

```
  char message[64];
  snprintf(message, sizeof(message), "T:%.2f H:%.2f", t, h);

  // ECC → Secreto compartido
  const struct uECC_Curve_t* curve = uECC_secp256r1();
  uint8_t sharedSecret[32];
  if (!uECC_shared_secret(peerPublicKey, privateKey, sharedSecret, curve)) {
    Serial.println("Error generando secreto");
    return;
  }

  // AES-128-CBC con sharedSecret como clave
  mbedtls_aes_context aes;
  uint8_t iv[16];
  for (int i = 0; i < 16; i++) iv[i] = esp_random() & 0xFF;

  size_t len = strlen(message);
  size_t paddedLen = ((len + 15) / 16) * 16;
  uint8_t input[64] = {0};
  uint8_t output[64] = {0};
  memcpy(input, message, len);

  mbedtls_aes_init(&aes);
  mbedtls_aes_setkey_enc(&aes, sharedSecret, 128);
  uint32_t start = micros();
  mbedtls_aes_crypt_cbc(&aes, MBEDTLS_AES_ENCRYPT, paddedLen, iv, input, output);
  uint32_t end = micros();
  mbedtls_aes_free(&aes);

  Serial.printf("Encriptado en %lu us\n", end - start);

  WiFiClient client;
  if (client.connect(server_ip, server_port)) {
    client.write(iv, 16);         // Enviar IV primero
    client.write(output, paddedLen); // Luego mensaje cifrado
    client.stop();
    Serial.println("Mensaje enviado.");
  }

  delay(5000);
}
```

Código del nodo central con el algoritmo ECC

```
#include <WiFi.h>
#include <WiFiServer.h>
#include <uECC.h>
#include <mbedtls/aes.h>
#include <Preferences.h>

const char* ssid = "";
const char* password = "";
const uint16_t server_port = 1234;

Preferences prefs;

WiFiServer server(server_port);

uint8_t privateKey[32];
uint8_t publicKey[64];

// Clave pública del emisor (ESP32 #1)
uint8_t peerPublicKey[64] = {
  0x23, 0xA4, 0x2C, 0x9B, 0x47, 0xE6, 0x15, 0xFA,
  0xB2, 0x0F, 0xF4, 0x64, 0xAC, 0xF1, 0x39, 0x8E,
  0x93, 0xE2, 0x01, 0x78, 0xF0, 0xCB, 0xDE, 0x13,
  0xAA, 0x4F, 0x88, 0xDC, 0x1C, 0x42, 0xE7, 0x36,
  0x51, 0xB9, 0x7D, 0xA1, 0xEF, 0xD3, 0x2A, 0x0C,
  0xD6, 0xF5, 0x68, 0x3B, 0x92, 0x40, 0x10, 0x72,
  0xD0, 0x9F, 0xA8, 0xD2, 0x1B, 0xBE, 0xFE, 0x5C,
  0xC7, 0x9A, 0x81, 0x69, 0x17, 0x03, 0xEC, 0x24
};

void generateECCKeys() {
  const struct uECC_Curve_t* curve = uECC_secp256r1();
  uECC_make_key(publicKey, privateKey, curve);
  prefs.begin("ecc", false);
  prefs.putBytes("priv", privateKey, 32);
  prefs.putBytes("pub", publicKey, 64);
  prefs.end();
}

bool loadECCKeys() {
  prefs.begin("ecc", true);
```

```
  if (prefs.isKey("priv") && prefs.isKey("pub")) {
    prefs.getBytes("priv", privateKey, 32);
    prefs.getBytes("pub", publicKey, 64);
    prefs.end();
    return true;
  }
  prefs.end();
  return false;
}

void setup() {
  Serial.begin(115200);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("WiFi conectado.");
  Serial.print("IP maestro: ");
  Serial.println(WiFi.localIP());
  if (!loadECCKeys()) {
    Serial.println("Generando claves ECC...");
    generateECCKeys();
  } else {
    Serial.println("Claves ECC cargadas.");
  }

  Serial.println("Clave pública del receptor:");
  for (int i = 0; i < 64; i++) Serial.printf("%02X", publicKey[i]);
  Serial.println();

  server.begin();
}

void loop() {
  WiFiClient client = server.available();
  if (client) {
    Serial.println("Cliente conectado. Recibiendo datos...");

    // Recibir IV (16 bytes) y mensaje cifrado
    uint8_t iv[16];
    while (client.available() < 16) delay(1);
    client.read(iv, 16);
```

```
  // Esperar hasta recibir al menos 16 más
  while (client.available() < 16) delay(1);
  size_t encLen = client.available();
  uint8_t encrypted[64] = {0};
  client.read(encrypted, encLen);

  client.stop();


  const struct uECC_Curve_t* curve = uECC_secp256r1();
  uint8_t sharedSecret[32];
  if (!uECC_shared_secret(peerPublicKey, privateKey, sharedSecret, curve)) {
    Serial.println("Error generando secreto compartido");
    return;
  }

  mbedtls_aes_context aes;
  uint8_t decrypted[64] = {0};
  mbedtls_aes_init(&aes);
  mbedtls_aes_setkey_dec(&aes, sharedSecret, 128);
  uint32_t start = micros();
  mbedtls_aes_crypt_cbc(&aes, MBEDTLS_AES_DECRYPT, encLen, iv, encrypted, decrypted);
  uint32_t end = micros();
  mbedtls_aes_free(&aes);

  Serial.printf("Descifrado en %lu us\n", end - start);
  Serial.print("Mensaje recibido: ");
  Serial.println((char*)decrypted);
 }
}
```

Código del nodo sensor con el algoritmo 3DES

```
#include <WiFi.h>
#include <DHT.h>
#include "tripledes.h"

#define DHTPIN 4
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

const char* ssid = "";
```

```
const char* password = "";
const IPAddress serverIP(192, 168, 1, 6);
const int port = 1234;

WiFiClient client;
uint8_t key[24] = {
  0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,
  0x09,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F,0x10,
  0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18
};
void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }
  Serial.println("WiFi conectado");
  client.connect(serverIP, port);

  tripledes_set_keys(key);
}
void loop() {
  if (!client.connected()) {
    Serial.println("Reintentando conexión...");
    client.stop();
    if (!client.connect(serverIP, port)) {
      Serial.println("Fallo al reconectar");
      delay(2000);
      return;
    }
    Serial.println("Conexión restablecida");
  }

  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  if (isnan(temp) || isnan(hum)) return;

  char payload[32];
  snprintf(payload, sizeof(payload), "T:%.1f H:%.1f", temp, hum);
```

```
  uint8_t input[8];
  strncpy((char*)input, payload, 8); // Solo los primeros 8 bytes

  uint8_t encrypted[8];
  unsigned long start = millis();
  tripledes_encrypt(input, encrypted);
  unsigned long end = millis();

  client.write(encrypted, 8);

  Serial.print("Cifrado enviado: ");
  for (int i = 0; i < 8; i++) Serial.printf("%02X ", encrypted[i]);
  Serial.printf("  Tiempo: %lu ms\n", end - start);

  delay(5000); // espera antes del siguiente envío
}
```

Código del nodo central con el algoritmo 3DES

```
#include <WiFi.h>
#include "tripledes.h"

const char* ssid = "";
const char* password = "";
WiFiServer server(1234);

uint8_t key[24] = {
  0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08,
  0x09,0x0A,0x0B,0x0C,0x0D,0x0E,0x0F,0x10,
  0x11,0x12,0x13,0x14,0x15,0x16,0x17,0x18
};
void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }
  Serial.println("WiFi conectado, esperando datos...");
  server.begin();
  Serial.print("IP maestro: ");
  Serial.println(WiFi.localIP());
```

```
  tripledes_set_keys(key);
}
void loop() {
  WiFiClient client = server.available();
  if (client && client.available() >= 8) {
    uint8_t encrypted[8], decrypted[8];
    client.read(encrypted, 8);
    unsigned long start = micros();
    tripledes_decrypt(encrypted, decrypted);
    unsigned long end = micros();
    Serial.print("Cifrado recibido: ");
    for (int i = 0; i < 8; i++) Serial.printf("%02X ", encrypted[i]);
    Serial.println();

    Serial.print("Texto descifrado: ");
    Serial.write(decrypted, 8); Serial.println();
    Serial.printf("Tiempo de descifrado: %lu ms\n\n", end - start);
  }
}
```

Código del circuito con el sensor INA 219

```
#include <Wire.h>
#include <Adafruit_INA219.h>

Adafruit_INA219 ina219;

float energiaTotal_mWh = 0.0;
unsigned long tiempoAnterior = 0;

void setup() {
  Serial.begin(115200);
  if (!ina219.begin()) {
    Serial.println("No se encontró el sensor INA219. Verifica la conexión.");
    while (1); // Se queda aquí si no detecta el sensor
  }
  tiempoAnterior = millis(); // Inicializar el tiempo
}

void loop() {
  float potencia = ina219.getPower_mW(); // Potencia instantánea en mW
```

```
  unsigned long tiempoActual = millis(); // Tiempo actual

  float intervaloSegundos = (tiempoActual - tiempoAnterior) / 1000.0;
  float energia = (potencia * intervaloSegundos) / 3600.0; // mWh en ese intervalo

  energiaTotal_mWh += energia;
  tiempoAnterior = tiempoActual;

  // Mostrar datos
  Serial.print("Potencia: ");
  Serial.print(potencia);
  Serial.println(" mW");

  Serial.print("Energía acumulada: ");
  Serial.print(energiaTotal_mWh);
  Serial.println(" mWh");

  Serial.println("----------------------");
  delay(1000);
}
```